

Court+ Ops Dashboard

Technical Specifications, Architecture & System Design Document

PLATFORM

Web — Static SPA

FRONTEND

React 18 · TypeScript ·
Tailwind v4

BACKEND

Supabase (PostgreSQL · RLS ·
Edge Functions)

STATE

React Hooks + Context
(useAuth)

HOSTING

GitHub Pages (public)

ANALYZER

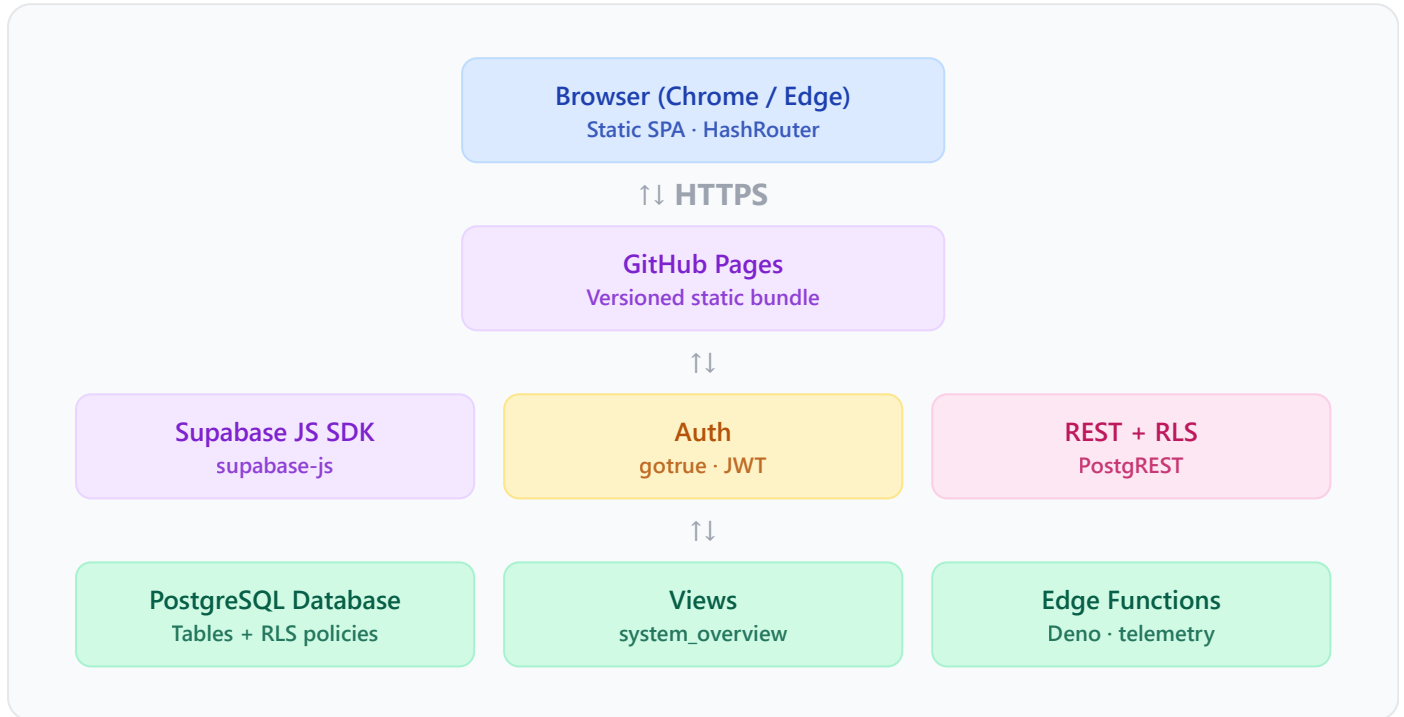
0 errors, 0 warnings

Comprehensive technical overview of the Court+ operations dashboard — a real-time observability, CI/CD, and court/coach management console for the Court+ team. Covers the system architecture, frontend design system, backend telemetry pipeline, Edge Function inventory, quality gates, CI/CD, and the production-readiness roadmap.

1 System Architecture Overview

The dashboard is a static single-page application with a **layered architecture**: a **React presentation layer** reads operational telemetry through a typed **API client**, which talks to **Supabase** over authenticated REST. All security-sensitive writes are routed through **Edge Functions** that use the service-role key and are protected by shared-secret or JWT gates.

System Topology



Data flow: the dashboard reads telemetry, logs, events, and pipeline runs via authenticated REST (RLS restricts reads to signed-in admins). Live activity and events are written by the mobile app through `ingest-events`, CI posts results through `ci-report`, and the optional 10-k demo drives `simulate-usage` — all Edge Functions running with the auto-injected service-role key and explicit gates.

2 Frontend Specifications (React)

The dashboard is a Vite + React 18 + TypeScript app styled with Tailwind CSS v4 and Recharts. Six routed pages, a shared design system, and a realtime data layer.

ROUTING Hash-based navigation

Uses HashRouter so deep links survive static hosting. Centralised route table with an auth guard: `#/login`, `#/overview`, `#/pipelines`, `#/logs`, `#/courts`, `#/coaches`. Unauthenticated users are redirected to `login`.

STATE Hooks + Context

Auth state in a `useAuth` context (`session`, `profile`, `isAdmin`). Data fetching with `useAsync` + `usePolling` hooks, a typed `api` client, and a shared `useDark` theme hook kept in sync across components.

DESIGN SYSTEM Theme & components

Emerald brand tokens via Tailwind `@theme`; light + custom dark theme (CSS-variable driven). Reusable primitives: `StatCard`, `StatusPill`, `Badge`, `Modal`, `ActivityFeed`, `Topbar`. Icons: **Phosphor**.

REALTIME Live data layer

Overview polls at **4s**, `ActivityFeed` at **5s**, `Pipelines & Logs` at **8s**. The request-volume chart, service health, Recent-errors table, and notifications repaint live without a page reload.

SIMULATION 10k-user demo toggle

A top-bar toggle drives `simulate-usage`: **start** (24h backfill of ~28k events + live ticks), **tick** (ongoing stream), **stop** (purges every `sim=true` row). While on, an honest **"Demo · simulated"** badge is shown so the data is never mistaken for real users.

AUTH Admin gating

Email/password sign-in against Supabase Auth; writes on `Courts/Coaches` are gated by `profiles.role = 'admin'`. A quick-admin sign-in button exists only in local dev — the credentials are excluded from the public bundle.

Live flow (Overview)

1 Poll operations data

`usePolling(() => api.systemOverview(), [], 4000)` → aggregates `request_metrics`, `system_logs`, and `bookings` through the `system_overview` view, refreshing the KPI cards (Success Rate, P95 latency, Bookings today, Errors) and the live chart every 4 seconds.

3 Backend, Telemetry & Data

Supabase provides Postgres storage, Row-Level Security, auth, and serverless Edge Functions. A telemetry pipeline ingests app logs and events; aggregated views feed the live dashboard.

Auth & Access Control

Built on **Supabase Auth (gotrue)**: email/password with persistent sessions and automatic token refresh. A profiles row tracks each user and carries the `role` column — `admin` unlocks management screens. Telemetry tables are **authenticated-select only**, so anonymous reads return empty and only signed-in admins can view live activity.

Core Database Schema

TABLE	KEY COLUMNS	RELATIONSHIPS	RLS
profiles	id, email, username, role	id → auth.users	ENABLED
courts	id, name, location, surface, price, is_active	—	ENABLED
coaches	id, name, specialty, rating, is_active	—	ENABLED
bookings	id, court_id, user_id, slot_id, status, total_price	court_id, user_id, slot_id	ENABLED
system_logs	id, level, service, message, context, user_id, app_version, sim	—	ENABLED
app_events	id, event, props, device, platform, tenant, sim	—	ENABLED
request_metrics	id, service, total, errors, p95_ms, period, sim	—	ENABLED
pipeline_runs	id, status, workflow_name, run_number	—	ENABLED
admin_audit_log	id, admin_id, action, target_table	—	ENABLED
system_overview (view)	success_rate, p95_latency_ms, bookings_today, errors_24h, requests_24h	aggregates metrics tables	VIEW

Telemetry Ingest Pipeline

The Flutter app streams RemoteLogger batches to `ingest-logs`, and EventTracker batches user actions (`screen_open`, `court_viewed`, `search_used`, `booking_created`) to `ingest-events`. Writes use the `service-role` key via Edge Functions; reads are `admins-only`. The live ActivityFeed and Logs pages surface this data within seconds.

1 Event buffering

`EventTracker.track(event, props)` buffers up to 50 events, then flushes in a batch POST to `/functions/v1/ingest-events` → `{"ok": true, "inserted": N}`. Silent no-op when not attached, so app tests are unaffected.

4 Edge Functions Inventory

Deployed as Deno serverless functions (`--no-verify-jwt`), each using the auto-injected service-role key with explicit authentication or shared-secret gates where they escalate privileges.

FUNCTION	PURPOSE	STATUS
<code>ingest-logs</code>	App log ingestion (RemoteLogger batches)	LIVE
<code>ingest-events</code>	App user-action event ingestion (EventTracker)	LIVE
<code>heartbeat</code>	Service health signal for the Overview (+ <code>request_metrics</code>)	LIVE
<code>ci-report</code>	Persists GitHub Actions pipeline results to <code>pipeline_runs</code>	LIVE · SECRET
<code>simulate-usage</code>	Toggleable 10k-user demo (start / tick / stop + purge)	LIVE · GATED
<code>create-admin</code>	Idempotent admin bootstrap + self-healing password	LIVE · GATED
<code>create-test-user</code>	Test-user bootstrap for QA sign-in	LIVE · GATED
<code>create-payment-intent</code>	Stripe PaymentIntent creation	WIP · FROZEN
<code>stripe-webhook</code>	Stripe webhook handler for payment confirmation	WIP · FROZEN

1 Gating model

`simulate-usage` requires a valid authenticated JWT (role authenticated, unexpired) and is CORS-enabled. `create-admin` / `create-test-user` require the shared `BOOTSTRAP_SECRET` header — without it they return **401**, so the released project cannot be used to mint admins or users. `ci-report` requires `CI_REPORT_SECRET`.

2 Simulation lifecycle

Start: `{ mode: 'start' }` — backfills ~28,000 realistic events across the past 24h tagged `sim=true`. **Tick:** live heartbeat + recent-window events every few seconds. **Stop:** `{ mode: 'stop' }` deletes every `sim=true` row, restoring only the authentic data.

5 Version Control, Quality & CI/CD

Monorepo containing both the Flutter client and the dashboard. Quality is enforced by static analysis with a fully green CI pipeline and a separate Pages deploy workflow.

GIT Branching model

Feature branches off `main`, reviewed via pull requests. `main` is the production branch and the source for both CI and the Pages deploy; no `develop/staging` branch yet.

ANALYSIS Static checks

`tsc --noEmit` → **0 errors** (dashboard). Flutter dart analyze → **0 errors, 0 warnings**, and `flutter test` → **123 passing in ~4s**.

ENV Configuration

Secrets live in a gitignored `.env` (`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`) and in Supabase secrets (`CI_REPORT_SECRET`, `QUICK_ADMIN_*`, `BOOTSTRAP_SECRET`). The quick-admin password is deliberately excluded from the public bundle.

PIPELINE GitHub Actions

CI: analyze + test + build (Flutter pinned to 3.38.4) and dashboard typecheck/build, then deploys Edge Functions and posts results back via `ci-report`. **Deploy Ops Dashboard:** builds and publishes the dashboard to GitHub Pages.

1 Deployment

Every push to `main` triggers the CI pipeline (static → test → APK build → report). The dashboard workflow builds with only the public `VITE_*` repo secrets and publishes to `medo962006.github.io/court_plus` — verified live, HTTP 200, all runs green.

6 Security, Release Status & Roadmap

Release state and open items, ordered by business impact. Each item maps a current limitation to a concrete remediation.

Security posture

AUTH Verified

Admin sign-in verified live; anonymous telemetry reads blocked by RLS; quick-admin credentials verified **absent** from the production bundle (`isAdmin` gates management writes).

EXPOSURE Gated

Privileged Edge Functions return **401** without their secret/JWT. Secrets are never committed; public payloads are minimal deploy-time `VITE_*` vars only.

Technical debt & roadmap

CRITICAL Payment integration (frozen / WIP)

`create-payment-intent` & the Stripe webhook are deployed but unconfigured (no `STRIPE_SECRET_KEY`). Payment telemetry is intentionally hidden from the dashboard.

Fix: add Stripe keys, wire confirm/refund, then surface payment events. **Estimate:** 2–3 weeks.

HIGH Realtime instead of HTTP polling

Live data is server-pollled at 4–8s intervals rather than pushed.

Fix: adopt `supabase-js` Realtime channels for the feed + timeline. **Estimate:** 4–5 days.

HIGH Real user traffic

Telemetry is sparse until the Court+ app ships to end users; the dashboard currently shines in demo mode.

Fix: ship to users, then the live feed, charts, and bookings counters populate organically.

MEDIUM Deploy automation for backend

The `deploy_backend` job is gated on `SUPABASE_ACCESS_TOKEN`.

Fix: set the token as a repo secret to enable fully automated migrations/function deploys.
Estimate: 1 day.

MEDIUM

Proactive alerting

Notifications are a manual poll of recent errors — no push alert on red runs or down services.

Fix: add alert rules (telegram/email) triggered by ci-report and heartbeat. **Estimate:** 2–3 days.

LOW

Bundle size & error tracking

Single JS chunk is ~950 kB; no Sentry integration yet.

Fix: code-split routes, adopt manualChunks, add Sentry. **Estimate:** 2 days.
